

ComfyUI & OpenWebUI Integration Guide

Table of Contents

1. [Initial Setup](#)
 2. [Multiple Workflows](#)
 3. [img2img Workflows](#)
 4. [Resources & Tutorials](#)
 5. [Troubleshooting](#)
-

Initial Setup

Step 1: Verify and Prepare ComfyUI

1. **Access your ComfyUI instance** at (`http://image.lmathes.cc:8188`) (or your server address)
 - If not running, start it via SSH: (`python main.py --listen 0.0.0.0 --port 8188`)
2. **Download required model:** (`z-image-turbo-fp8-aio.safetensors`)
 - Place in ComfyUI's (`models/checkpoints`) folder
3. **Load your workflow:**
 - Click "Load" and upload your JSON workflow file
 - Test it by clicking "Queue Prompt"
4. **Enable Dev Mode:**
 - Click the gear icon near "Queue Prompt"
 - Check "Enable Dev mode Options"
5. **Export API format:**
 - Click "Save (API Format)" to download (`workflow_api.json`)
 - This converts the workflow to API structure for OpenWebUI

Step 2: Configure OpenWebUI

1. Open OpenWebUI (e.g., (`http://localhost:3000`))
2. Go to **Admin Panel > Settings > Images** tab
3. Set **Image Generation Engine** to **ComfyUI**
4. Enter **ComfyUI API URL:** (`http://image.lmathes.cc:8188/`) (include trailing slash)

For Docker installations, add environment variables:

```
yaml
COMFYUI_BASE_URL=http://image.lmathes.cc:8188/
ENABLE_IMAGE_GENERATION=True
```

5. Toggle **Image Generation (Experimental)** ON
6. Test the connection (should show success message)

Step 3: Upload and Map the Workflow

In the Images tab:

1. **Upload** the `workflow_api.json` file
2. **Configure node mappings** (based on your workflow):
 - **Positive Prompt Node ID:** 2 (CLIPTextEncode node)
 - **Negative Prompt Node ID:** 3 (negative prompt node)
 - **Seed Node ID:** 5 (KSampler node)
 - **Steps Node ID:** 5 (input: "steps")
 - **CFG Node ID:** 5 (input: "cfg")
3. Save the changes

Step 4: Generate Images

1. Start a chat in OpenWebUI
 2. Use prompt: `@image Generate an image of [your description]`
 3. Example: `@image A fluffy white cat in a similar style to the poodle workflow`
 4. The generated image will appear inline
-

Multiple Workflows

OpenWebUI supports multiple ComfyUI workflows simultaneously:

How to Set Up Multiple Workflows

1. **Create different workflows in ComfyUI** for different purposes:
 - Text-to-image
 - Image-to-image
 - Upscaling
 - Inpainting
 - Style transfer
2. **Export each as API format** (`workflow_api.json`)
3. **Upload to OpenWebUI** with descriptive names:
 - "workflow-txt2img-realistic"
 - "workflow-img2img-style"
 - "workflow-upscale-4x"
4. **Switch between workflows:**
 - Select from dropdown (version-dependent)
 - Use specific commands/prefixes in prompts
 - Set one as default, switch manually when needed

Workflow Organization Tips

- Name workflows clearly by function
 - Keep a backup of all workflow files
 - Document which models each workflow requires
 - Test each workflow independently before uploading
-

img2img Workflows

Creating an img2img Workflow in ComfyUI

Basic img2img Pipeline:

LoadImage → VAE Encode → KSampler → VAE Decode → SaveImage

Key Components:

1. **LoadImage Node:** Receives the input image from OpenWebUI
2. **VAE Encode:** Converts image to latent space
3. **KSampler:**
 - Set denoise to less than 1.0 (typically 0.3-0.8)
 - Lower denoise = subtle changes
 - Higher denoise = major transformation
4. **VAE Decode:** Converts back to image
5. **SaveImage:** Returns result to OpenWebUI

Export and Configure img2img

1. Build your img2img workflow in ComfyUI
2. Enable Dev Mode and export as API format
3. Upload to OpenWebUI
4. **Map the Image Input Node ID** (the LoadImage node)
 - Find the node ID in `workflow_api.json`
 - Set this in OpenWebUI's workflow configuration

Using img2img in OpenWebUI

1. Upload an image in the chat
2. Add your prompt describing modifications
3. OpenWebUI sends both image and prompt to ComfyUI
4. Image goes to LoadImage node, text to prompt node

img2img Workflow Examples

Style Transfer:

- Input: Photo
- Denoise: 0.6-0.7
- Prompt: "oil painting style, vibrant colors"

Upscaling:

- Use upscale nodes (Upscale Image, etc.)
- Lower denoise (0.3-0.5) to preserve details

Sketch to Image:

- Input: Rough sketch
- Higher denoise (0.7-0.9)
- Detailed prompt describing final image

Inpainting:

- Use LoadImageMask nodes
- Specify areas to modify
- Rest of image remains unchanged

Denoise Strength Guide

- **0.1-0.3:** Minimal changes, refinement only
 - **0.4-0.6:** Moderate changes, style adjustments
 - **0.7-0.9:** Major transformation, creative reimagining
 - **1.0:** Full generation (essentially text-to-image)
-

Resources & Tutorials

Official Documentation

OpenWebUI Official Docs - ComfyUI Integration

- URL: <https://docs.openwebui.com/features/image-generation-and-editing/comfyui/>
- Coverage: Setup, workflow export, node mapping, image generation and editing
- Best for: Comprehensive setup instructions

Step-by-Step Tutorials

Noted.lol Tutorial - Connecting ComfyUI to OpenWebUI

- URL: <https://noted.lol/connect-comfyui-to-open-webui/>
- Coverage: Step-by-step connection guide, workflow export, configuration
- Best for: Beginners setting up for the first time

Open-WebUI.com Guide

- URL: <https://open-webui.com/comfyui/>
- Coverage: Basic integration, FLUX model setup
- Best for: Quick reference guide

Community Tools & Examples

ComfyUI img2img Tool by owoDra

- URL: https://openwebui.com/t/owodra/comfyui_img2img_tool
- GitHub: <https://github.com/owoDra/comfyui-img2img-tools.git>
- Features: Dedicated img2img tool with reference image generation

ComfyUI Official img2img Examples

- URL: https://comfyanonymous.github.io/ComfyUI_examples/img2img/
- Coverage: Official examples showing LoadImage → VAE Encode → KSampler workflow

GitHub Gist - Workflow Examples

- URL: <https://gist.github.com/thinkyhead/841aa574b6bfbf51a4f23d78d2c7c8b5>
- Content: Example workflows for OpenWebUI integration

YouTube Search Suggestions

While specific OpenWebUI + ComfyUI tutorials are limited, search for:

- "OpenWebUI ComfyUI tutorial"
- "ComfyUI img2img workflow"
- "ComfyUI API integration"
- "ComfyUI workflow basics"

Community Support

- **OpenWebUI Discord:** For integration-specific questions
- **OpenWebUI GitHub:** Bug reports and feature requests
- **ComfyUI Reddit:** r/comfyui for workflow examples
- **CivitAI:** Pre-made workflows and models

Troubleshooting

Connection Errors

Issue: Cannot connect to ComfyUI server

Solutions:

- Ensure no firewall blocks port 8188
- Test with: `curl http://image.lmathes.cc:8188/` from OpenWebUI host
- Add `--enable-cors-header` to ComfyUI startup if CORS issues occur
- Verify server is running: `python main.py --listen 0.0.0.0 --port 8188`

Model Not Found

Issue: "Model not found" error

Solutions:

- Check model path in workflow (e.g., `(Z-image\z-image-turbo-fp8-aio.safetensors)`)
- Fix backslashes to forward slashes on Linux: `(Z-image/z-image-turbo-fp8-aio.safetensors)`
- Verify model exists in `(ComfyUI/models/checkpoints/)`
- Re-download model if corrupted

Node Mapping Issues

Issue: OpenWebUI throws errors about node IDs

Solutions:

- Inspect `(workflow_api.json)` (it's a dict of node IDs with inputs)
- Find correct node IDs in the JSON file
- Adjust mappings in OpenWebUI to match
- Common node types to look for:
 - CLIPTextEncode (positive/negative prompts)
 - KSampler (seed, steps, cfg, denoise)
 - LoadImage (for img2img)

Slow Generation

Issue: Images take too long to generate

Solutions:

- Reduce steps from 100 to 20-50 in the workflow
- Use faster models (turbo, LCM, SDXL Lightning)
- Lower resolution in workflow settings
- Check server resources (CPU/GPU usage)

Image Quality Issues

Issue: Generated images are low quality or incorrect

Solutions:

- Increase steps (50-100 for higher quality)
- Adjust CFG scale (7-9 for better prompt adherence)
- Verify sampler settings (euler, DPM++, etc.)
- Check if correct model is loaded
- For img2img: adjust denoise strength

Security Considerations

For Remote Servers:

- Use `--auth` flags for ComfyUI authentication
- Set up HTTPS if server is publicly accessible
- Use firewall rules to restrict access
- Consider VPN for sensitive workflows
- Don't expose port 8188 directly to internet

Docker-Specific Issues

Issue: OpenWebUI can't reach ComfyUI in Docker

Solutions:

- Use host network mode: `--network host`
- Use `host.docker.internal` instead of `localhost`
- Ensure containers are on same Docker network
- Check environment variables are set correctly

Workflow Customization Tips

- **For variations:** Modify workflow in ComfyUI, re-export API, re-upload
 - **Monitor logs:** Check ComfyUI console for errors during API calls
 - **Version workflows:** Keep numbered versions of successful workflows
 - **Test incrementally:** Make small changes and test before complex modifications
-

Advanced Tips

Optimizing Workflows

1. **Use LoRAs** for specific styles without retraining
2. **Implement ControlNet** for precise control over composition
3. **Chain multiple workflows** for complex effects
4. **Use checkpoint switching** for different art styles
5. **Implement upscaling nodes** for high-resolution outputs

Batch Processing

- Set up workflows with batch size nodes
- Use queue system for multiple generations
- Implement seed arrays for variations

Performance Tuning

- Use FP8/FP16 models for faster generation
 - Enable TensorRT or xformers optimization
 - Adjust batch sizes based on VRAM
 - Use faster samplers (DPM++ SDE, Euler A)
-

Quick Reference: Common Node IDs

When setting up node mappings, look for these common patterns in your workflow_api.json:

Function	Node Type	Typical ID Range
Positive Prompt	CLIPTextEncode	2-6
Negative Prompt	CLIPTextEncode	2-6
Seed	KSampler	3-10
Steps	KSampler	3-10
CFG	KSampler	3-10
Denoise	KSampler	3-10
Image Input	LoadImage	1-15
Model Loader	CheckpointLoaderSimple	1-5

Note: IDs vary by workflow structure. Always verify in your specific workflow_api.json file.

Appendix: Example workflow_api.json Structure

json

```
{
  "1": {
    "inputs": {
      "ckpt_name": "z-image-turbo-fp8-aio.safetensors"
    },
    "class_type": "CheckpointLoaderSimple"
  },
  "2": {
    "inputs": {
      "text": "your positive prompt here",
      "clip": ["1", 1]
    },
    "class_type": "CLIPTextEncode"
  },
  "3": {
    "inputs": {
      "text": "blurry, low quality",
      "clip": ["1", 1]
    },
    "class_type": "CLIPTextEncode"
  },
  "5": {
    "inputs": {
      "seed": 123456,
      "steps": 100,
      "cfg": 7.5,
      "sampler_name": "euler",
      "scheduler": "simple",
      "denoise": 1.0
    },
    "class_type": "KSampler"
  }
}
```

Document Version: 1.0

Last Updated: January 2026

Compatible with: OpenWebUI (latest), ComfyUI (stable release)

For the most current information, always refer to the official documentation links provided above.